

第3章 分支结构

3.1 关系表达式

- 作用：比较两个值
- 关系运算符：
 - < （小于）
 - <= （小于等于）
 - > （大于）
 - >= （大于等于）
 - == （等于）
 - != （不等于）
- = 表示赋值运算， == 表示相等
- 关系表达式： <expression> <relation operator> <expression>
 - 结果为 bool （ true 或 false ）
- 结合方式：左结合
 - 如： $-2 < -1 < 0 \Leftrightarrow (-2 < -1) < 0 \Leftrightarrow \text{true} < 0$ ，所以 $-2 < -1 < 0$ 的结果为 false ，与代数不同

3.2 逻辑表达式

- 作用：处理多个 bool 变量为1个 bool 变量
- 逻辑运算符：
 - ! （一元运算符，非）
 - || （二元运算符，或）
 - && （二元运算符，与）
- 运算：执行 bool 变量的运算，格式如下：
 - !<expression>
 - <expression 1> && <expression 2>
 - <expression 1> || <expression 2>
- 结果如下表所示

p	q	p&&q	p q	!p
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true

p	q	p&&q	p q	!p
false	false	false	false	true

- 编译器对执行过程的优化：部分编译器在编译时会作出以下优化
 - `<expression 1> && <expression 2>` 中，若 `<expression 1>` 为 `false`，则跳过 `<expression 2>` 直接返回 `false`
 - `<expression 1> || <expression 2>` 中，若 `<expression 1>` 为 `true`，则跳过 `<expression 2>` 直接返回 `true`
 - 具体优化与否要根据编译器版本而定，**因此不推荐在 `<expression 2>` 中包含赋值语句**
- 优先级：从上到下依次降低
 - `!`
 - `*`，`/`，`%`
 - `+`，`-`
 - `<`，`>`，`<=`，`>=`
 - `==`，`!=`
 - `&&`
 - `||`

3.3 if 语句

3.3.1 if 语句的形式

- 格式：
 - `if (<expression>) {<codeblock 1>}`
 - 若 `<expression>` 为 `true`，则执行 `<codeblock 1>` 的所有语句
 - 否则跳过 `<codeblock 1>`，即什么也不做
 - `if (<expression>) {<codeblock 1>} else {<codeblock 2>}`
 - 若 `<expression>` 为 `true`，则执行 `<codeblock 1>` 的所有语句
 - 否则执行 `<codeblock 2>` 的所有语句
- 子句：`<codeblock i>` 可以为单个语句（此时可以省略 `{}`），也可以是多个语句
 - `<expression>` 为 `true` 执行的语句块称为 `then` 子句
 - `<expression>` 为 `false` 执行的语句块称为 `else` 子句

3.3.2 if 语句的嵌套

- if 语句的嵌套：`then`子句或`else`子句含有 `if` 语句的情况
- `else` 的配对：向前寻找最近的没有与 `else` 配对的 `if`
 - 但不会到 `{}` 内部与 `if` 配对
- 用缩进关系表示 `if` 与 `else` 的配对情况是良好的编程习惯

3.3.3 条件表达式

- 条件运算符： `? :` （三元运算符同时出现，一起使用）
- 条件表达式： `<expression> ? <value 1> : <value 2>`
 - 若 `<expression>` 为 `true` ，则结果为 `<value 1>`
 - 若 `<expression>` 为 `false` ，则结果为 `<value 2>`

3.4 switch 语句

- 格式：

```
switch(<expression>)  
{  
    case <constant 1> :  
        <codeblock 1>  
    case <constant 2> :  
        <codeblock 1>  
    /* Other case statement */  
    case <constant n> :  
        <codeblock n>  
    default :  
        <codeblock n+1>  
}
```

- 执行：
 - i. 计算 `<expression>` 的值
 - ii. 将结果从 `<constant 1>` 开始匹配
 - iii. 若匹配到了 `<constant i>` ，则匹配结束，从 `<codeblock i>` **开始执行**
 - iv. 若未匹配到任何常量，则匹配结束，从 `<codeblock n+1>` **开始执行**
 - `default : <codeblock n+1>` 也可直接省略，此时若未匹配到任何常量，则会直接退出 `switch` 语句块
- `break` 关键字：在 `switch` 语句中，若执行到 `break;` 语句，则会直接跳出 `switch` 语句
 - 适用于在执行到下一个 `case` 或 `default` 前取消执行后面的语句块的情况
- `<expression>` 的要求：应为整型表达式（包括结果能转化为整型的表达式）